



UNIVERSITI PUTRA MALAYSIA

“BERILMU BERBAKTI”

CCS4202 – DESIGN AND ANALYSIS OF ALGORITM GROUP ASSIGNMENT

MALAYSIA DROUGHT: WATER OPTIMIZATION

LECTURER’S NAME:

DR. NORWATI MUSTAPHA

NAME	MATRIC
MUHAMMAD ASLAM ROFI BIN MOHD RASU	230405
AQIL QAYYIM BIN ABDUL KHALID	228096
HERMAWAN RYAN IQBAL	226745
ABDULLAH ALI IMRAN BIN IFFLI EFENDY	SA00076

SUBMITTED DATE:

11/6/2026

CCS4202/CSC4202 Design and Analysis of Algorithm

Second Semester, 2025/2026

Learning outcomes:

1. Search for and organize information in the form of an strengthen algorithm for a computer problem as the following: (LL)
 - a. Analyse the problem to suggest a solution by choosing a suitable algorithm design technique.
 - b. Develop skills to reason about and prove properties of algorithms such as their correctness and running time.

Instructions:

1. General instructions:
 - a) Create an original scenario (you can be inspired by examples on the internet or used in the course, but the detailing of the story should be created fresh by your team of 3 or 4 people) that requires an optimal solution.
 - b) Explain why finding an optimal solution for this scenario is important.
 - c) Review the suitability of sorting, D&C, DP, greedy and graph algorithms as a solution paradigm for the chosen problem by stating their strengths and weaknesses.
 - d) Design the algorithm to solve the problem and explain the idea of your algorithm paradigm by emphasising which part needs **recurrence** and the function for the **optimization**.
 - e) Define the algorithm specification.
 - f) Develop a Java program.
 - g) Provide an analysis of the **algorithm's correctness** as well as **time complexity** (best, average and worst time) by using asymptotic notation.
 - h) Develop an online portfolio (using google sites or github or google colab or any suitable tool) with the following steps/contents:
 - i. Illustrate the problem.
 - ii. Explain your algorithm paradigm and show the **pseudocode**. You may provide your code in the portfolio if you wish.
 - iii. Demonstrate your program and describe the output.
 - iv. Describe the algorithm analysis.
 - i) Deliver a presentation in week 14.
 - j) Submit the following through Putrablast before the presentation:
 - i. Link to your online portfolio
 - ii. A zip file of your codes
 - iii. Filled project progress (refer [APPENDIX](#))
2. Please make sure each member's workload is fairly distributed and good project management is exercised. The weight of the project is 20%. The evaluations are based on filled progress monitoring (10 marks), online portfolio in github (50 marks) and presentation (20 marks).
3. Follow the following algorithm specification steps:

- Problem definition based on the chosen scenario (tell your story by choosing a geographical setting, type of disaster, damage impact, highlight the importance of AAD in the scenario and provide illustrations, state the goal and expected output to support the decision making)
- Development of a model for the chosen scenario (state the data type, state the objective function and constraints, provide examples and other requirements based on the scenario such as objective, space, time or value constraints.
- Specification of an algorithm (state which topic and algorithm have you selected and why, include comparison of several other options and discuss the suitability of your proposed solution)
- Designing an algorithm (provide a pseudocode and/or flowchart and use illustrations to help you)
- Checking the correctness of an algorithm (asymptomatic, recurrence)
- Analysis of an algorithm (growth of function for worst, best, average analysis)
- Implementation of an algorithm.
- Program testing (provide a demo based on your story of the chosen scenario)
- Documentation through online portfolio.

4. Tips for completing the project

- a) Understanding of the AAD topics including examples of problems and list of algorithms that each topic covers is a MUST to ensure you can design the solution based on the instructions given.
- b) The “How Might We (HMW) technique” launches brainstorms by asking questions that seed your idea. Eg, HMW maximize the profit from building on this location by optimising the selection from a list of property options? From this question, your group may discuss potential situations and solutions. Then, discuss the possible algorithms to be used for this problem.
- c) You need to work collaboratively and be encouraged to use various materials around you as your reference (please make sure you include a good bibliography list in your documentation). During discussion, you may use ideas reviewing techniques such as identifying the “Pluses, Potential, Concerns, Options (PPCO)”.
 Pluses: What are (at least) three things you like about the idea?
 Potentials: What are (at least) three good things that might result if the idea were implemented?
 Concerns: What are some concerns you have about the idea (phrased as a question starting with “How to...” or “How might...”)
 Options or Overcome the concerns: What are some ideas you have for how to fix the concerns you just noted?
 Or, you may use the Strength, Weakness, Opportunities, Threats (SWOT) technique.
- d) Time management and each member’s dedication is the key to the group’s success.
- e) Use your creativity and critical thinking skills to design the algorithms and communicate it well in written and verbal format.

INTRODUCTION

Background & Problem Definition

Northern Malaysia, specifically the "rice bowl" states of Kedah and Perlis, is facing an unprecedented and severe prolonged drought. As the central reservoir levels drop critically low, the region faces a catastrophic risk of widespread crop failure. The damage impact extends beyond local farming communities, threatening the national food security and economic stability of the country.

The Goal & Need for Optimization

To mitigate this disaster, the state government has mobilized a fleet of water distribution trucks. However, the total available water is strictly limited and cannot meet the demands of all farming districts simultaneously. Our goal is to develop an algorithmic model that distributes this centralized water supply optimally, prioritizing regions based on crop vulnerability, absolute need, and logistical distance.

Finding an optimal solution for this scenario is absolutely critical. A suboptimal distribution means wasting irreplaceable water on low-priority or inaccessible areas, resulting in the avoidable death of critical farmlands. To support rapid decision-making under these high-pressure constraints, our team evaluated three distinct algorithmic paradigms: Greedy (Fractional Knapsack), Dynamic Programming (0/1 Knapsack), and Dijkstra's Algorithm (Graph Routing).

WHAT FILES IS NEEDED

FILES	FUNCTION
DijkstraRouter.java	Calculates shortest water transport routes using graph logic.
DynamicProgramming.java	Maximizes drought priority scores using 0/1 knapsack logic.
GreedyAllocator.java	Quickly allocates fractional water based on priority ratios.
OptimizationAlgorithm.java	Outdated interface template replaced by the new version.
WaterDistributionAlgoritm.java	Standardizes inputs and outputs for all your algorithms.
Node.java	Stores critical data for each farming district region.
Region.java	Outdated data model completely replaced by Node class.
layout.fxml	Defines visual interface layouts when using JavaFX SceneBuilder.
MainApp.java	Launches the main application window and button actions.
MainController.java	Links FXML visual elements to your Java code.

THE JAVAFX VISUAL OUTPUT

CCS4202 Assignment 1 - Water Optimization

Northern Malaysia Drought: Water Optimization

Compare Greedy, Dynamic Programming, and Dijkstra-based routing strategies

Total Available Water:

Region Name	Water Required	Priority Score
Alor Setar (High Priority)	10	9.5
Kangar	5	6.0
Sungai Petani	12	8.0
Baling	8	7.5
Pendang	7	5.0

Dataset updated successfully.

Execution Time: 3039300 ns

```
--- Results for: Greedy Algorithm ---
Total Water Capacity: 25 units

Kangar           | Needed: 5 | Allocated: 5 | Priority: 6.00
Alor Setar (High Priority) | Needed: 10 | Allocated: 10 | Priority: 9.50
Baling           | Needed: 8 | Allocated: 8 | Priority: 7.50
Pendang          | Needed: 7 | Allocated: 2 | Priority: 5.00
Sungai Petani    | Needed: 12 | Allocated: 0 | Priority: 8.00

Summary:
Water Used: 25 / 25
Total Priority Score Saved: 28.0
```

By using JavaFX, we can easily put input and change the available water, water required, the priority score, and test the same output with different algorithm to see the execution time, water used, each region allocation and total priority score.

EXPLANATION OF CODES

GREEDY ALGORITHM

```
1  public void distributeWater(List<Node> nodes, int totalWater) {
2      // 1. Sort by Priority-to-Need ratio (Highest to Lowest)
3      nodes.sort((a, b) -> Double.compare(
4          b.priorityScore / b.waterRequired,
5          a.priorityScore / a.waterRequired
6      ));
7
8      // 2. Greedy Fractional Allocation
9      for (Node node : nodes) {
10         int allocated = Math.min(node.waterRequired, totalWater);
11         node.waterAllocated = allocated;
12         totalWater -= allocated;
13
14         if (totalWater <= 0) break;
15     }
16 }
```

nodes.sort((a, b) -> Double.compare(b.priorityScore / b.waterRequired, a.priorityScore / a.waterRequired));

This is the absolute heart of the Greedy Fractional Knapsack algorithm. It calculates the "value per unit of water" for each region by dividing its priority score by its water requirement. It then sorts the entire list in descending order, pushing the most critical, high-yield regions to the front of the line.

❏ **int allocated = Math.min(node.waterRequired, totalWater);**

Just like in the Dijkstra condensation, this single line replaces the bulky if/else block. It determines the maximum water the region can receive: either exactly what they asked for, or whatever fraction of water is left in the reservoir—whichever number is smaller.

❏ **node.waterAllocated = allocated; totalWater -= allocated;**

This physically records the water transfer. It assigns the calculated allocated amount to the specific region's variable and deducts that exact amount from the remaining centralized supply.

❏ **if (totalWater <= 0) break;**

A vital optimization check. Once the total water reaches zero, this instantly breaks the loop. There is no need to keep calculating or looping through the remaining lower-priority regions because the reservoir is completely empty.

DYNAMIC PROGRAMMING

```
1 public void distributeWater(List<Node> nodes, int totalWater) {
2     int n = nodes.size();
3     double[][] dp = new double[n + 1][totalWater + 1];
4
5     // 1. Build the Optimization Table
6     for (int i = 1; i <= n; i++) {
7         Node node = nodes.get(i - 1);
8         for (int w = 0; w <= totalWater; w++) {
9             if (node.waterRequired <= w) {
10                 dp[i][w] = Math.max(dp[i - 1][w],
11                                     dp[i - 1][w - node.waterRequired] + node.priorityScore);
12             } else {
13                 dp[i][w] = dp[i - 1][w];
14             }
15         }
16     }
17
18     // 2. Backtrack to assign water
19     for (int i = n; i > 0 && totalWater > 0; i--) {
20         if (dp[i][totalWater] != dp[i - 1][totalWater]) {
21             Node node = nodes.get(i - 1);
22             node.waterAllocated = node.waterRequired;
23             totalWater -= node.waterRequired;
24         }
25     }
26 }
```

double[][] dp = new double[n + 1][totalWater + 1];

This initializes your 2D memory table. The rows represent the number of regions considered so far, and the columns represent every possible water capacity from 0 up to your maximum limit.

dp[i][w] = Math.max(dp[i - 1][w], dp[i - 1][w - node.waterRequired] + node.priorityScore);

This is the core "recurrence relation" (which hits your assignment requirement perfectly). It calculates the absolute maximum priority score by testing two choices:

1. *Leave it*: Carry over the best score from the previous regions (dp[i - 1][w]).
2. *Take it*: Subtract the water this region needs, find the best score for that remaining water, and add this region's priority score. It takes whichever option yields the higher score.

if (dp[i][totalWater] != dp[i - 1][totalWater])

This is the start of the **backtracking** phase. The DP table only calculates the *maximum score*, it doesn't remember *who* got the water. This line checks if the score changed compared to the row directly above it. If the score is different, it means the current region was officially included in the optimal solution.

node.waterAllocated = node.waterRequired; totalWater -= node.waterRequired;

Once the if statement confirms the region was part of the optimal solution, this physically allocates their required water quota and subtracts it from the remaining capacity so the loop can continue tracking backward.

DJIKSTRA ROUTER

```
1 public void distributeWater(List<Node> nodes, int totalWater) {
2     // 1. Graph Definition
3     int[][] graph = {
4         {0, 4, 0, 0, 8}, {4, 0, 6, 0, 2}, {0, 6, 0, 3, 0},
5         {0, 0, 3, 0, 5}, {8, 2, 0, 5, 0}
6     };
7     int n = graph.length;
8     int[] dist = new int[n];
9     boolean[] visited = new boolean[n];
10
11     // 2. Initialization
12     Arrays.fill(dist, Integer.MAX_VALUE);
13     dist[0] = 0;
14
15     // 3. Dijkstra's Core Logic (Shortest Path)
16     for (int i = 0; i < n - 1; i++) {
17         int u = -1;
18         // Find nearest unvisited node
19         for (int j = 0; j < n; j++) {
20             if (!visited[j] && (u == -1 || dist[j] < dist[u])) u = j;
21         }
22
23         visited[u] = true;
24
25         // Edge Relaxation
26         for (int v = 0; v < n; v++) {
27             if (!visited[v] && graph[u][v] != 0 &&
28                 dist[u] + graph[u][v] < dist[v]) {
29                 dist[v] = dist[u] + graph[u][v];
30             }
31         }
32     }
33
34     // 4. Map Distances and Sort
35     for (int i = 0; i < nodes.size(); i++)
36         nodes.get(i).distanceFromReservoir = dist[i];
37     nodes.sort((a, b) -> Integer.compare(a.distanceFromReservoir, b.distanceFromReservoir));
38
39     // 5. Greedy Allocation
40     for (Node node : nodes) {
41         int allocated = Math.min(node.waterRequired, totalWater);
42         node.waterAllocated = allocated;
43         totalWater -= allocated;
44         if (totalWater <= 0) break;
45     }
46 }
```

Arrays.fill(dist, Integer.MAX_VALUE); dist[0] = 0;

This sets all initial distances to infinity, except for the source node (the reservoir), which is set to 0 so the algorithm starts there.

if (!visited[j] && (u == -1 || dist[j] < dist[u])) u = j;

This inline loop replaces your entire findNearestUnvisitedNode method. It simply scans the array to find the unvisited node (u) with the smallest current distance.

if (!visited[v] && graph[u][v] != 0 && dist[u] + graph[u][v] < dist[v])

This is the crucial **edge relaxation** step. It checks if traveling through the current node u provides a shorter path to neighbor v than any previously found path. If yes, it updates dist[v].

nodes.sort((a, b) -> Integer.compare(a.distanceFromReservoir, b.distanceFromReservoir));

A concise lambda expression that sorts the regions from closest to furthest based on the shortest paths Dijkstra just calculated.

int allocated = Math.min(node.waterRequired, totalWater);

This drastically simplifies your allocation if/else block. It safely assigns either the exact amount the region needs, or whatever fraction of water is left—whichever number is smaller.

THE SET VALUES

Region Name	Water Required	Priority Score
Alor Setar (High Priority)	10	9.5
Kangar	5	6.0
Sungai Petani	12	8.0
Baling	8	7.5
Pendang	7	5.0

RESULTS

GREEDY ALGORITHM

DYNAMIC PROGRAMMING

DJKSTRA ROUTING

TOTAL WATER = 10

Kangar	Needed: 5 Allocated: 5	Alor Setar (High Priority)	Needed: 10 Allocated: 10	Alor Setar (High Priority)	Needed: 10 Allocated: 10
Alor Setar (High Priority)	Needed: 10 Allocated: 5	Kangar	Needed: 5 Allocated: 0	Kangar	Needed: 5 Allocated: 0
Baling	Needed: 8 Allocated: 0	Sungai Petani	Needed: 12 Allocated: 0	Pendang	Needed: 7 Allocated: 0
Pendang	Needed: 7 Allocated: 0	Baling	Needed: 8 Allocated: 0	Sungai Petani	Needed: 12 Allocated: 0
Sungai Petani	Needed: 12 Allocated: 0	Pendang	Needed: 7 Allocated: 0	Baling	Needed: 8 Allocated: 0

35300 ns	17800 ns	95300 ns
WATER USED = 10/10	WATER USED = 10/10	WATER USED = 10/10
TOTAL PRIORITY = 15.5	TOTAL PRIORITY = 9.5	TOTAL PRIORITY = 9.5
Achieved the highest priority score (15.5) by fractionally splitting the limited water between two regions.	Ran the fastest but yielded a lower priority score (9.5) because it allocated all water to a single region.	Ran the slowest and mirrored the exact same lower priority allocation as DP.

TOTAL WATER = 40

Kangar	Needed: 5 Allocated: 5	Alor Setar (High Priority)	Needed: 10 Allocated: 10	Alor Setar (High Priority)	Needed: 10 Allocated: 10
Alor Setar (High Priority)	Needed: 10 Allocated: 10	Kangar	Needed: 5 Allocated: 5	Kangar	Needed: 5 Allocated: 5
Baling	Needed: 8 Allocated: 8	Sungai Petani	Needed: 12 Allocated: 12	Pendang	Needed: 7 Allocated: 7
Pendang	Needed: 7 Allocated: 7	Baling	Needed: 8 Allocated: 8	Sungai Petani	Needed: 12 Allocated: 12
Sungai Petani	Needed: 12 Allocated: 10	Pendang	Needed: 7 Allocated: 0	Baling	Needed: 8 Allocated: 6

38500 ns	34200 ns	58700 ns
WATER USED = 40/40	WATER USED = 35/40	WATER USED = 40/40
TOTAL PRIORITY = 36	TOTAL PRIORITY = 31	TOTAL PRIORITY = 36
Maximized the priority score (36) by using all 40 units of water perfectly.	Left 5 units of water completely unused due to its strict non-fractional rules, resulting in a lower priority score (31).	Successfully used all water and matched the high priority score (36), but remained the slowest algorithm.

TOTAL WATER = 10 000 000

Kangar	Needed: 5 Allocated: 5	Alor Setar (High Priority)	Needed: 10 Allocated: 10	Alor Setar (High Priority)	Needed: 10 Allocated: 10
Alor Setar (High Priority)	Needed: 10 Allocated: 10	Kangar	Needed: 5 Allocated: 5	Kangar	Needed: 5 Allocated: 5
Baling	Needed: 8 Allocated: 8	Sungai Petani	Needed: 12 Allocated: 12	Pendang	Needed: 7 Allocated: 7
Pendang	Needed: 7 Allocated: 7	Baling	Needed: 8 Allocated: 8	Sungai Petani	Needed: 12 Allocated: 12
Sungai Petani	Needed: 12 Allocated: 12	Pendang	Needed: 7 Allocated: 7	Baling	Needed: 8 Allocated: 8

TIME = 41600 ns	TIME = 243 153 400 ns (2.4 seconds)	TIME = 73500 ns
WATER USED = 42 / 10000000	WATER USED = 42 / 10000000	WATER USED = 42 / 10000000
TOTAL PRIORITY = 36	TOTAL PRIORITY = 36	TOTAL PRIORITY = 36
Scaled perfectly, handling the massive capacity almost instantly (41,600 ns) while meeting all needs.	Suffered a catastrophic time complexity failure because its processing time explodes as knapsack capacity increases.	Scaled efficiently, handling the large capacity quickly (73,500 ns) and successfully meeting all needs.

CONCLUSION FOR EACH ALGORITHM

1. The Greedy Algorithm (Fractional Knapsack)

Conclusion:

The Greedy approach proved to be the most computationally efficient and high-yielding algorithm for resource allocation, provided that the resource is completely divisible. By sorting the farming districts based on a "priority-to-need" ratio, it consistently generated the highest possible survival score for the crops. It demonstrated flawless scalability, handling massive datasets with almost zero performance degradation.

Strengths:

- **Unmatched Time Complexity:** Because the algorithm only requires sorting the array and making a single pass through the data, its time complexity is bounded by $O(N \log N)$. This was evident when the capacity jumped from 40 to 10,000,000 units, and the execution time barely shifted (only 41,600 ns).
- **Maximized Efficiency:** In the 10-unit test, it achieved a priority score of 15.5, significantly higher than the other algorithms. By allowing fractional distribution, it guarantees that not a single drop of water is wasted.
- **Low Memory Overhead:** It evaluates the data strictly in place without needing to build complex two-dimensional arrays or graphs, making it highly memory efficient.

Weaknesses:

- **Ignores Discrete Logistics:** The core weakness of this algorithm is its assumption that water can be perfectly divided. If the state government of Kedah is deploying water via standard 10,000-liter trucks, you cannot dispatch "half a truck" to Baling and "half a truck" to Pendang. If the resource is indivisible, the Greedy approach falls apart in the real world.
- **No Geographic Awareness:** It distributes based purely on mathematical priority, completely ignoring how far these districts are from the central reservoir.

2. Dynamic Programming (0/1 Knapsack)

Conclusion:

Dynamic Programming offers the absolute mathematical optimum for discrete, indivisible resource allocation. However, your results clearly expose its fatal flaw: pseudo-polynomial time complexity. While it is incredibly reliable for small-scale, rigid logistical constraints (like whole water trucks), it becomes practically unusable when scaling up to massive, continuous resource volumes.

Strengths:

- **Perfect for Discrete Constraints:** When water must be delivered in indivisible units, DP guarantees the highest possible priority score without breaking the rules.
- **High Accuracy at Low Scales:** For smaller datasets (like the 10-unit and 40-unit tests), it processes rapidly (17,800 ns and 34,200 ns) and provides a highly dependable decision-making matrix.

Weaknesses:

- **Catastrophic Scalability (Time Complexity):** The time complexity of this algorithm is $O(N \times W)$, where W is the total water capacity. This is why the 10,000,000-unit test took a massive 2.4 seconds (243,153,400 ns). The algorithm had to calculate and compare millions of redundant subproblems.
- **Severe Space Complexity:** To solve the 10,000,000-unit problem, the algorithm had to construct a 2D array in memory with millions of columns. In a production environment with limited RAM, this would cause a fatal memory overflow.
- **Resource Wasting:** In the 40-unit test, the DP algorithm left 5 units of water completely unused (allocating only 35/40) because the remaining water did not perfectly match the exact requirements of the remaining unallocated districts.

3. Dijkstra Routing (Graph Algorithm)

Conclusion:

Dijkstra's algorithm serves a fundamentally different purpose than the knapsack algorithms. While DP and Greedy decide *who* gets the water, Dijkstra is necessary to determine *how* to physically transport it there. Your results show that while it carries more initial computational overhead, it scales beautifully and provides a highly realistic solution for physical disaster relief logistics.

Strengths:

- **Geographic Realism:** It is the only algorithm that factors in the physical constraints of the real world. By mapping the roads between Alor Setar, Kangar, and Sungai Petani, it ensures that water trucks take the shortest, most cost-effective paths.
- **Excellent Scalability:** Unlike DP, Dijkstra's performance did not degrade severely when the water capacity increased. Even at 10,000,000 units, it processed the routes in a swift 73,500 ns. Its time complexity, generally $O(V^2)$ or $O(E + V \log V)$ with a priority queue, depends on the number of regions, not the volume of water.

Weaknesses:

- **Slower Baseline Processing:** For the smaller datasets (10 and 40 units), Dijkstra was noticeably the slowest algorithm (95,300 ns and 58,700 ns). This is due to the overhead of constructing the graph nodes, edges, and maintaining the priority queue in Java.
- **Doesn't Solve the Value Problem Alone:** Dijkstra finds the shortest path, but it doesn't inherently optimize for the *priority score* of the crops. In a perfect software model, you would actually need to pair Dijkstra (for routing) with a Greedy or DP algorithm (for allocation) to get a complete solution.

FINAL CONCLUSION

Summary of Findings

Through our rigorous testing across varying resource capacities (10 units, 40 units, and 10,000,000 units), it became clear that no single algorithm offers a flawless standalone solution; each has distinct strengths and critical weaknesses tied to the nature of the resource constraints.

- **For Continuous Allocation:** The **Greedy Algorithm** proved to be the absolute best choice if the water can be divided into infinitely small fractions. It executed with unmatched speed $O(N \log N)$ and perfectly maximized the survival priority score at every scale without wasting a single drop.
- **For Discrete Allocation:** **Dynamic Programming** provided the mathematical guarantee of maximum value when water must be delivered in strict, indivisible units (like whole trucks). However, our tests revealed its fatal flaw: pseudo-polynomial time and space complexity. When scaled to real-world industrial volumes (10,000,000 units), the algorithm suffered massive performance degradation, proving it is unsuitable for large-scale continuous resources.
- **For Physical Logistics:** **Dijkstra's Algorithm** was the only approach that anchored our data in geographical reality. While Greedy and DP can calculate *who* deserves the water, Dijkstra is fundamentally necessary to figure out the most cost-effective way to drive it there. It handled massive data scaling efficiently, though it carried higher baseline computational overhead for smaller sets.

The Proposed Optimal Solution

Based on our analysis, the most effective real-world solution for disaster relief in Kedah and Perlis is a **hybrid approach**.

To survive a crisis of this scale, the government should first employ the **Greedy Algorithm** to rapidly allocate the water quotas, assuming bulk transport allows for fractional distribution. Once the quotas are mathematically locked in, the system must immediately feed those winning destination nodes into **Dijkstra's Algorithm** to generate the optimal delivery routes for the physical trucks. By combining the $O(N \log N)$ allocation speed of Greedy with the spatial awareness of Dijkstra, we achieve a solution that is computationally lightning-fast, mathematically optimal, and geographically realistic.

ALL SOURCE CODE = <https://github.com/aslamrofi/DAA-GROUP-ASG>

REFERENCES

Frontiers in Environmental Science. (2023). Water resources optimal allocation model for coordinating regional multi-level water resources managers' interests. *Frontiers in Environmental Science*. <https://doi.org/10.3389/fenvs.2023.1152296>

Huang, C.-L., & Hsu, N.-S. (2026). Analytical trade-off simulation-optimization of drought-resistant water supply allocation strategies under various demands using a multi-objective cross-watersheds network-flow model. *EGU General Assembly 2026*. <https://doi.org/10.5194/egusphere-egu26-2158>

MDPI. (2025). Efficient Approach for the Sectorization of Water Distribution Systems: Integrating Graph Theory and Binary Particle Swarm Optimization. *Sustainability*. <https://www.mdpi.com/2071-1050/17/9/4231>

Waseem, M. (2026). How can hybrid AI-physics models improve drought prediction and water allocation in semi-arid regions? *ResearchGate*. [https://www.researchgate.net/post/How can hybrid AI-physics models improve drought prediction and water allocation in semi-arid regions](https://www.researchgate.net/post/How_can_hybrid_AI-physics_models_improve_drought_prediction_and_water_allocation_in_semi-arid_regions)

ALL SOURCE CODE = <https://github.com/aslamrofi/DAA-GROUP-ASG>

END OF GROUP ASSIGNMENT

Initial Project Plan (week 10, submission date: 5 June 2026)

Group Name		AIR SELANGOR	
Members			
Name		Email	Phone number
MUHAMMAD ASLAM ROFI BIN MOHD RASU		230405@student.upm.edu.my	0148077505
AQIL QAYYIM BIN ABDUL KHALID		228096@student.upm.edu.my	0173117966
HERMAWAN RYAN IQBAL		226745@student.upm.edu.my	01140662236
ABDULLAH ALI IMRAN BIN IFFLI EFENDY		sa00076@student.upm.edu.my	0108716138
Problem scenario description	Northern Malaysia, specifically the "rice bowl" states of Kedah and Perlis, is experiencing a severe and prolonged drought. Central reservoir levels are critically low, and local farming districts face an imminent risk of widespread crop failure. The state government must deploy a limited fleet of water distribution trucks to multiple regions		
Why it is important	This crisis directly threatens national food security and the economic survival of rural farming communities. Water is strictly limited; a suboptimal distribution means wasting resources on low-priority or inaccessible areas, resulting in the avoidable death of highly vulnerable and critical farmlands.		
Problem specification	Data Types: A list of Node objects representing farming districts, containing variables for waterRequired, priorityScore, waterAllocated, and distanceFromReservoir. Constraints: The sum of water allocated cannot exceed the totalReservoirCapacity. Water may be continuous (liters) or discrete (whole trucks). Objective Function: Maximize the overall crop survival rate, defined as the sum of Water Allocated x Priority Score, while identifying the shortest physical transport routes		
Potential solutions	1. Greedy Algorithm (Fractional Knapsack): Allocates continuous water rapidly based on a priority-to-need ratio. 2. Dynamic Programming (0/1 Knapsack): Maximizes priority scores for discrete, indivisible water truck deliveries. 3. Dijkstra's Algorithm (Graph Routing): Calculates the most cost-effective and shortest physical transport routes from the central reservoir to the districts.		
Sketch (framework, flow, interface)	Framework: Model-View-Controller (MVC) architecture separating pure algorithmic logic from the UI. Flow: The user inputs the total available water capacity -> selects an algorithm -> the system computes the optimal distribution and tracks execution time -> the UI updates with the results. Interface: A JavaFX desktop application featuring a top control panel with three execution buttons (Greedy, DP, Dijkstra) and a monospaced text-area dashboard that displays a clear, tabular summary of regions, water needed, water allocated, and execution time in nanoseconds.		

Project Proposal Refinement (week 11, submission date: 12 June 2026)

Group Name		AIR SELANGOR	
Members			
		Name	Role
		MUHAMMAD ASLAM ROFI BIN MOHD RASU	Programmer
		AQIL QAYYIM BIN ABDUL KHALID	Designer for JavaFX
		HERMAWAN RYAN IQBAL	Paperwork and slides
		ABDULLAH ALI IMRAN BIN IFFLI EFENDY	Researcher and data analyst
Problem statement		The agricultural sector in Northern Malaysia (Kedah and Perlis) is facing a critical water shortage due to a prolonged drought. With a limited centralized water supply, the government struggles to distribute water efficiently to various farming districts to prevent catastrophic crop failure and ensure food security.	
Objectives		1. To design an algorithmic model that optimally allocates a limited water supply across multiple farming districts. 2. To maximize the overall crop survival rate based on regional priority scores. 3.To calculate the shortest and most cost-effective physical routing for water distribution trucks. 4. To analyze and compare the time complexity, efficiency, and real-world viability of Greedy, Dynamic Programming, and Dijkstra's algorithms	
Expected output		A JavaFX desktop application that accepts total water capacity as an input, executes three distinct distribution algorithms, and outputs a tabular dashboard showing the allocated water per region, total priority score saved, and the exact nanosecond execution time for comparative analysis.	
Problem scenario description		Northern Malaysia, specifically the "rice bowl" states of Kedah and Perlis, is experiencing a severe drought. The central reservoir is critically low. The government must deploy water trucks to multiple regions, but the supply is strictly insufficient to meet all demands simultaneously.	
Why it is important		This crisis directly threatens national food security and the economic survival of rural farming communities. Water is strictly limited; a suboptimal distribution means wasting resources on low-priority or physically inaccessible areas, resulting in the avoidable death of highly vulnerable and critical farmlands.	
Problem specification		Data Model: Node objects representing districts with properties: waterRequired, priorityScore, waterAllocated, and distanceFromReservoir. Constraints: sum of Water Allocated x Total Reservoir Capacity. Water is treated as either continuous (liters) or discrete units (whole trucks). Objective Function: Maximize the survival score, defined as sum of Water Allocated x Priority Score, while minimizing transport distance.	
Potential solutions		1. Greedy Algorithm (Fractional Knapsack): Provides fast, $O(N \log N)$ continuous water allocation based on a priority-to-need ratio. 2. Dynamic Programming (0/1 Knapsack): Provides the absolute optimal allocation for discrete, indivisible units, though it requires heavy $O(NW)$ computational time. 3. Dijkstra's Algorithm (Graph Routing): Calculates the shortest physical transport routes from the reservoir to the districts in $O(V^2)$ time.	

Sketch (framework, flow, interface)	Framework: Model-View-Controller (MVC) architecture separating pure algorithmic logic from the UI. Flow: The user inputs the total available water capacity -> selects an algorithm -> the system computes the optimal distribution and tracks execution time -> the UI updates with the results. Interface: A JavaFX desktop application featuring a top control panel with three execution buttons (Greedy, DP, Dijkstra) and a monospaced text-area dashboard that displays a clear, tabular summary of regions, water needed, water allocated, and execution time in nanoseconds.	
Methodology	Milestone	Time
	Find a disaster in Malaysia and make a scenario	Week 10
	Find the solution and researching the algorithms. Start doing the barebone of code and do testing.	Week 11
	Debugging the coding and turning normal java into JavaFX	Week 12
	Analysis of all the algorithm and decide the best one	Week 13
	Presentation in class	Week 14

Project Progress (Week 10 – Week 14)

Milestone 1	Scenario Finalization & Core Algorithms				
Week	Week 11				
Description	We finalized the Northern Malaysia drought scenario and established the core Java architecture. The JavaFX user interface layout was drafted, and the foundational data models (Node.java) and interfaces were created. We successfully programmed and unit-tested the first two algorithms: Greedy (Fractional Knapsack) and Dynamic Programming (0/1 Knapsack).				
Role	Member 1	Member 2	Member 3	Member 4	
	ASLAM	AQIL	RYAN	IYAN	
	Set up JavaFX MainApp, created the shared Node model, and managed the MVC folder structure.	Coded GreedyAllocator.java and conducted basic fractional allocation testing.	Coded DynamicProgramming.java and handled the 2D array recurrence table logic.	Formulated the test datasets and drafted the theoretical Big-O time complexity proofs for the algorithms	

Milestone 2	Graph Routing, UI Integration & Testing			
Week	Week 13			
Description	The final graph algorithm (Dijkstra) was completed and mapped to the specific farming districts (Alor Setar, Kangar, etc.). All three algorithms were successfully linked to the JavaFX UI buttons. We conducted stress tests comparing time complexity across varying water capacities (10 to 10,000,000 units) and began assembling the results for the online GitHub portfolio.			
Role	Member 1	Member 2	Member 3	Member 4
	ASLAM	AQIL	RYAN	IYAN
	Developed DijkstraRouter.java, built the adjacency matrix, and tested shortest-path routing.	Connected algorithm outputs to the UI buttons and formatted the text-area dashboard.	Compiled the execution time data and drafted the final GitHub portfolio documentation.	Built the required online GitHub portfolio, uploaded the codebase, structured the final comparative analysis charts, and created the PowerPoint slides for the Week 14 presentation.